

Stratified RMST Boosting for Subgroup Identification

Augmenting the Value Function with Prognostic Strata

Yue Shentu & Natasha

May 28, 2026

Revised v2: corrected gradient scaling

Contents

1	Introduction	2
2	Notation	2
3	Prognostic Stratification	2
4	Stratified Training	3
4.1	Value Function	3
4.2	Analytic Gradient	3
4.3	Implementation	4
5	Simulation Results	5
5.1	Complex Boundary Scenarios (Zhang et al. 2020)	5
5.2	Gradient Combination Attempts	6

1 Introduction

This document defines a stratified version of RMST-based subgroup boosting (“RMST CAV-Boost”). The original method learns a soft subgroup assignment $p_i \in [0, 1]$ (the probability that patient i belongs to the treatment-preferring subgroup) by optimizing a value function through gradient boosting trees. The proposed refinement replaces the original pooled value function with one computed within prognostic strata and aggregated.

We focus on **stratified training**, which replaces the value function used during boosting with the stratified version. The gradient is modified to incorporate stratum-specific RMST differences. This is the version evaluated in the simulations below.

2 Notation

For each subject $i = 1, \dots, N$, we observe:

- $\mathbf{Z}_i$: p -vector of baseline covariates;
- $A_i \in \{0, 1\}$: treatment (1 = active, 0 = control);
- $U_i = \min(T_i, C_i, \tau)$: observed time;
- $\tilde{\delta}_i = \mathbf{1}\{\min(T_i, \tau) \leq C_i\}$: event within τ before censoring.

The restricted mean survival time (RMST) at horizon τ is $\mu(\tau) = \int_0^\tau S(t) dt$, estimated by the area under the Kaplan–Meier curve.

3 Prognostic Stratification

A prognostic score s_i is a univariate summary of $\mathbf{Z}_i$ that predicts survival under the control condition. Because treatment is randomized, we can use all patients (both arms) to estimate s_i as long as treatment is excluded from the predictor set.

We use a cross-fitted elastic net Cox model to estimate s_i , balancing the need to handle $p \approx n/10$ covariates while avoiding overfit within each fold. The implementation:

1. Partition the training data into $F = 5$ random folds.
2. For fold $f = 1, \dots, F$:
 - (a) Fit `cv.glmnet( $\mathbf{Z}$ , Surv( $T, C$ ), family="cox", alpha=0.5)` on the out-of-fold observations.
 - (b) Predict the linear predictor s_i for in-fold observations at `lambda.min`.
3. Collect the out-of-fold predictions $\hat{s}_i$ for all training patients.
4. Define strata by K quantiles of $\hat{s}_i$.

Cross-fitting ensures that the stratum assignment for each patient is independent of its own data, eliminating the training-set overlap bias that would arise from fitting and predicting on the same observations. The elastic net penalty (mixing parameter $\alpha = 0.5$, cross-validated λ) automatically selects among the 52 covariates and shrinks noisy coefficients, which is essential when the Cox

model is proportional-hazards misspecified (e.g., under the quadratic prognostic structure of Zhang scenarios S4–S6).

Stratify s_i into K groups by its quantiles. Let $\mathcal{S}_k$ be the index set of stratum k , with $n_k = |\mathcal{S}_k|$ patients. For $K = 4$, these are quartiles; $K = 6$ and $K = 8$ are used in sensitivity analyses.

4 Stratified Training

In this mode, the value function used during XGBoost training is itself stratified. This changes the gradient that each tree fits, potentially altering the learned p_i .

4.1 Value Function

The stratified value function mirrors the original CAVBoost structure exactly, but applied within each stratum. Recall that the original value function is:

$$V_{\text{orig}} = \left(\sum_i p_i \right) \hat{\Delta}^{(1)} - \left(\sum_i (1 - p_i) \right) \hat{\Delta}^{(2)},$$

where $\hat{\Delta}^{(1)}$ uses p_i -weighted KM (subgroup 1, treatment-preferring) and $\hat{\Delta}^{(2)}$ uses $(1 - p_i)$ -weighted KM (subgroup 2, control-preferring).

The stratified version applies the same logic within each stratum k . Define within-stratum subgroup 1 KM (p -weighted) and subgroup 2 KM ($(1-p)$ -weighted), giving two RMST differences per stratum:

$$\begin{aligned} d_k^{(1)} &= \hat{\mu}_{1,k}^{(1)}(\tau) - \hat{\mu}_{0,k}^{(1)}(\tau), \\ d_k^{(2)} &= \hat{\mu}_{1,k}^{(2)}(\tau) - \hat{\mu}_{0,k}^{(2)}(\tau), \end{aligned}$$

where $\hat{\mu}_{a,k}^{(1)}(\tau)$ is the p -weighted KM RMST within stratum k , arm a , and $\hat{\mu}_{a,k}^{(2)}(\tau)$ is the $(1 - p)$ -weighted equivalent.

The stratified value function is:

$$V_{\text{strat}}(\mathbf{p}) = \sum_{k=1}^K W_k d_k^{(1)} - \sum_{k=1}^K (n_k - W_k) d_k^{(2)}$$

where $W_k = \sum_{i \in \mathcal{S}_k} p_i$ and $n_k = |\mathcal{S}_k|$.

This is the same *difference of deltas* format as the original, but the KM computations are restricted to each stratum. No normalization by N is applied, matching the scale convention of the original.

4.2 Analytic Gradient

The gradient mirrors the original CAVBoost derivation. For patient j in stratum k , taking the derivative of each term in V_{strat} with respect to $\eta_j = \log(p_j/(1 - p_j))$:

$$\begin{aligned} \textbf{Term 1: } \frac{\partial}{\partial \eta_j} [W_k d_k^{(1)}] &= p_j(1 - p_j) d_k^{(1)} + W_k \frac{\partial d_k^{(1)}}{\partial \eta_j} \\ \textbf{Term 2: } -\frac{\partial}{\partial \eta_j} [(n_k - W_k) d_k^{(2)}] &= p_j(1 - p_j) d_k^{(2)} - (n_k - W_k) \frac{\partial d_k^{(2)}}{\partial \eta_j} \end{aligned}$$

The chain rule $\partial d_k^{(g)}/\partial \eta_j = p_j(1 - p_j) \partial d_k^{(g)}/\partial p_j$ gives the full gradient:

$$\frac{\partial V_{\text{strat}}}{\partial \eta_j} = p_j(1 - p_j) \left[d_k^{(1)} + d_k^{(2)} + W_k \frac{\partial d_k^{(1)}}{\partial p_j} - (n_k - W_k) \frac{\partial d_k^{(2)}}{\partial p_j} \right].$$

The XGBoost loss function minimizes, so the gradient passed to the tree learner is $g_j = -\partial V_{\text{strat}}/\partial \eta_j$.

The influence functions. The terms $\partial d_k^{(1)}/\partial p_j$ and $\partial d_k^{(2)}/\partial p_j$ are the derivatives of the within-stratum p -weighted and $(1 - p)$ -weighted RMST differences with respect to patient j 's subgroup probability. They are computed by the same influence-function machinery as the original CAVBoost (the `hazard_inc` function in the R code), evaluated on the events within stratum k only. The per-stratum computation is K times faster than the full-sample version because each stratum has roughly $n_k \approx N/K$ patients.

Block-diagonal structure. A critical property: $\partial d_k^{(g)}/\partial p_j = 0$ for $j \notin \mathcal{S}_k$, because patient j does not contribute to the KM within stratum k . This means the gradient for patient j depends only on patients in the same stratum. The XGBoost regression tree must therefore learn stratum-relevant splits from the feature vector $\mathbf{Z}_j$, relying on the fact that the prognostic score s_i (used to define strata) is a function of $\mathbf{Z}_i$.

Connection to the original. The original CAVBoost gradient (from `rmst_cavboost_clean.R`) is:

$$g_j^{(\text{orig})} = -p_j(1 - p_j) \left[\hat{\Delta}^{(1)} + \left(\sum_i p_i \right) \frac{\partial \hat{\Delta}^{(1)}}{\partial p_j} + \hat{\Delta}^{(2)} - \left(\sum_i (1 - p_i) \right) \frac{\partial \hat{\Delta}^{(2)}}{\partial p_j} \right].$$

The stratified version is identical in structure, but with each term replaced by its stratum-level analogue:

$$\begin{aligned} \hat{\Delta}^{(1)} &\leftrightarrow d_k^{(1)}, & \hat{\Delta}^{(2)} &\leftrightarrow d_k^{(2)}, \\ \sum_i p_i &\leftrightarrow W_k, & \sum_i (1 - p_i) &\leftrightarrow n_k - W_k, \\ \frac{\partial \hat{\Delta}^{(1)}}{\partial p_j} &\leftrightarrow \frac{\partial d_k^{(1)}}{\partial p_j}, & \frac{\partial \hat{\Delta}^{(2)}}{\partial p_j} &\leftrightarrow \frac{\partial d_k^{(2)}}{\partial p_j}. \end{aligned}$$

The implementation is a direct modification of the original: the KM/hazard-increment computation loops over strata instead of the full sample.

4.3 Implementation

The algorithm mirrors the original CAVBoost, but the KM and influence computations loop over strata:

1. Pre-compute cross-fitted prognostic score $\hat{s}_i$ via 5-fold elastic net Cox (`glmnet`, $\alpha = 0.5$) and stratum assignments $\mathcal{S}_k$ by K quantiles. These are fixed throughout boosting.
2. Initialize $\eta_i = 0$ ($p_i = 0.5$).
3. For each boosting round:

- (a) Compute $p_i = 1/(1 + e^{-\eta_i})$.
 - (b) For each stratum k :
 - i. Compute p -weighted KM within $\mathcal{S}_k$, get $d_k^{(1)}$ and influence $\partial d_k^{(1)}/\partial p_j$.
 - ii. Compute $(1 - p)$ -weighted KM within $\mathcal{S}_k$, get $d_k^{(2)}$ and influence $\partial d_k^{(2)}/\partial p_j$.
 - iii. Assemble gradient for patients in $\mathcal{S}_k$: $g_j = -p_j(1 - p_j)[d_k^{(1)} + d_k^{(2)} + W_k \partial d_k^{(1)}/\partial p_j - (n_k - W_k) \partial d_k^{(2)}/\partial p_j]$.
 - (c) Fit a regression tree to $\{g_j, \mathbf{Z}_j\}$.
 - (d) Update $\eta_i \leftarrow \eta_i + \eta_{\text{boost}} \times \text{tree}(\mathbf{Z}_i)$.
4. After training: $\hat{G}_i = \mathbf{1}\{p_i > 0.5\}$.

This is a direct adaptation of the original CAVBoost gradient, replacing the global weighted KM with per-stratum weighted KM. The R functions from `rmst_cavboost_clean.R`—in particular the hazard increment and sorted-data machinery—are reused without modification; only the loop structure changes.

5 Simulation Results

All classification metrics are computed on a held-out test set of the same size as the training set. Three methods are compared: original CAVBoost, stratified CAVBoost ($K = 4$, cross-fitted elastic net Cox prognostic score), and virtual twin (separate survival random forests (ranger, 200 trees, min node size 10, RMST difference via trapezoidal integration).

5.1 Complex Boundary Scenarios (Zhang et al. 2020)

The Zhang et al. (2020) SubgroupBoost paper defines four scenarios with increasing boundary complexity, using 52 covariates (50 noise, 2 predictive S_1, S_2), $N_{\text{train}} = 500$, $N_{\text{test}} = 2000$, $\tau = 30$, and Setting 2 prognostic strength ($\beta_{Z_{1:4}} = 0.4$). The prognostic score is estimated via 5-fold cross-fitted Cox PH, so the within-stratum gradient computation uses out-of-fold predictions and avoids training-set overlap.

Results from 30 Monte Carlo reps:

Scenario (boundary)	Hold-out AUC			Gain (Orig $\rightarrow$ Strat)
	Original	Stratified	VT	
S1: Linear ($S_1 = 0$)	0.757	0.880	0.956	+0.123
S3: U-shaped (S_1)	0.844	0.932	0.509	+0.088
S4: Enclave ($S_1 \times S_2$)	0.778	0.764	0.508	−0.014
S5: S-shaped (S_1 , disjoint)	0.805	0.883	0.595	+0.078

The key findings are:

- **Virtual twin collapses on nonlinear boundaries** (S3: 0.509, S4: 0.508, S5: 0.595). The per-arm linear Cox PH cannot capture non-monotone or interaction-based treatment effect surfaces.

- **Stratified CAVBoost excels when the subgroup boundary is smooth relative to prognosis** (S1: +12.3 AUC points, S3: +8.8, S5: +7.8). The cross-fitted prognostic stratification aligns with smoothly varying treatment effects while the out-of-fold prediction eliminates the training-set overlap bias.
- **Stratification is neutral on disjoint boundaries** (S4 Enclave: -0.014 , within Monte Carlo error). When the true subgroup consists of disconnected regions in covariate space, the Cox PH-based prognostic score cannot capture the structure, but the original global gradient remains competitive.
- **Original CAVBoost is robust** across all scenarios (0.76–0.84 AUC), making it a reliable all-purpose baseline.

These results suggest that cross-fitted stratified CAVBoost is consistently beneficial on smooth or monotone boundaries and never catastrophically worse than the original. The approach is recommended as the default when the treatment-effect boundary is believed to be continuous in the prognostic space.

5.2 Gradient Combination Attempts

Two approaches to combining the original and stratified gradients were evaluated: alternating rounds (original on odd rounds, stratified on even) and variance-gated blending (α_k derived from Greenwood variance per stratum). Both failed on the complex boundary scenarios (AUC 0.55–0.60), because the two gradient functions are minimizing different estimands and their per-observation directions anti-correlate. A holdout model-selection rule (train both models, pick by held-out RMST gain) was also tested; it over-selected the stratified model on S4 and S5 (where the original has a slight edge), demonstrating that 150-patient holdouts lack the power to reliably discriminate between the two methods.